

AI Career Assistant

Complete Knowledge Base

30 Topics · 6 Categories · Interview Prep · Resume Building · Backend Technologies · CS Fundamentals

Generated: October 08, 2026

Table of Contents

■ Backend

- Python Backend Development
- REST APIs
- Databases & SQL
- Authentication & Authorization
- Docker & Containerisation
- Git & Version Control
- Linux & Command Line
- FastAPI & SQLAlchemy
- PostgreSQL & Database Management

■ CS Fundamentals

- DSA — Data Structures
- DSA — Algorithms
- Object Oriented Programming
- OS & System Concepts
- Computer Networking
- DSA
- System Design

■ Career

- Salary
- Skills
- Placement
- Study Plan
- Career Switch
- Open Source Contribution
- Certifications
- Freelancing & Remote Work

■ Frontend

- React & Frontend Development

■ Interview

- Interview Preparation
- Technical Interview
- HR Interview
- System Design Interview
- Aptitude & Logical Reasoning
- Interview

■ Resume

- Resume Building
- Resume Projects Section
- LinkedIn Profile
- Cover Letter
- Resume

Category: Interview

INTERVIEW

01. Interview Preparation

Keywords: *interview · preparation · prepare · round · how to crack · interview tips*

Complete interview preparation roadmap:

Phase 1 — Technical Foundation (8 weeks)

- Week 1–3: DSA — arrays, strings, hash maps, two pointer, sliding window
- Week 4–5: Trees, graphs, BFS/DFS, heaps
- Week 6–7: Dynamic programming, backtracking, greedy
- Week 8: System design basics (for SDE-2+ roles)

Phase 2 — Practice (4 weeks)

- Solve 2–3 LeetCode problems daily
- Do at least 2 full mock interviews per week
- Review every wrong answer within 24 hours
- Practice explaining your solution out loud (interviewers assess communication)

Phase 3 — Interview Week

- Day before: Revise your resume line by line
- Prepare 3 STAR stories for HR round
- Research the company: product, tech stack, recent news
- Sleep 8 hours — performance matters more than last-minute revision

On the day:

- Read every problem fully before writing code
- Clarify constraints and edge cases first
- Start with a brute force, then optimise
- Explain your thought process as you code

■ Resources

- NeetCode — Interview Roadmap — <https://neetcode.io/roadmap>
- Pramp — Free Mock Interviews — <https://pramp.com>
- InterviewBit — <https://interviewbit.com>
- GeeksforGeeks Interview Prep — <https://geeksforgeeks.org/company-interview-corner>

INTERVIEW

02. Technical Interview

Keywords: *technical interview · coding interview · coding round · technical round · online assessment · oa*

Technical interview / coding round guide:

Online Assessment (OA):

- Typically 2–3 problems in 60–90 minutes
- Focus on: arrays, strings, hash maps, two pointer, basic graphs
- Practice on HackerRank, Codechef, LeetCode contest mode
- Read problem constraints first — they hint at the expected approach

Technical Interview (Live Coding):

- Step 1 — Understand: restate the problem in your own words
- Step 2 — Examples: walk through 2–3 test cases including edge cases
- Step 3 — Brute force first: state TC/SC, then say 'can we do better?'
- Step 4 — Optimise: think aloud about patterns
- Step 5 — Code cleanly with meaningful variable names
- Step 6 — Test: trace through your code with an example

Most common patterns asked:

1. Two pointer / sliding window
2. Hash map for frequency counting
3. BFS/DFS on trees and graphs
4. Binary search on answer
5. DP (longest subsequence, knapsack variants)

Companies to practice by: Google (graphs, DP), Amazon (trees, arrays), Microsoft (trees, recursion), Infosys/TCS (basic arrays, strings)

■ Resources

- LeetCode — Company Tag Problems — <https://leetcode.com/problemset>
- HackerRank Practice — <https://hackerrank.com/domains/algorithms>
- Striver's SDE Sheet — <https://takeuforward.org/strivers-a2z-dsa-course>

INTERVIEW

03. HR Interview

Keywords: *hr interview · hr round · behavioural · behavioral · soft skills · star method · tell me about yourself · weakness*

HR / Behavioural interview preparation:

Tell Me About Yourself (2-minute formula):

Present: 'I am a final year B.Tech CSE student at [college].'
Past: 'I have interned at X and built Y project.'
Future: 'I'm looking for a role where I can work on Z.'

STAR Method for behavioural questions:

S — Situation: Set the scene
T — Task: Your specific responsibility

A – Action: What YOU did (not your team)

R – Result: Quantify the outcome

Top 10 HR questions + how to approach them:

1. Tell me about yourself → 2-min structured pitch
2. Greatest strength → Pick relevant, give a STAR example
3. Biggest weakness → Real weakness + what you're doing about it
4. Why this company? → Research their product and tech stack
5. Where do you see yourself in 5 years? → Aligns with role growth
6. Describe a challenge you overcame → STAR story
7. Describe a team conflict → Focus on resolution
8. Why should we hire you? → Skills + culture fit + enthusiasm
9. Do you have any questions? → Always ask 2–3 thoughtful questions
10. Salary expectations? → Research market, give a range, not a fixed number

Prepare at least 5 STAR stories that can cover: failure, success, teamwork, leadership, and a difficult technical problem.

■ Resources

- STAR Method Explained — Indeed — <https://indeed.com/career-advice/interviewing/how-to-use-the-star-interview-response-technique>
- Top HR Questions — Glassdoor — <https://glassdoor.com/blog/common-interview-questions>

INTERVIEW

04. System Design Interview

Keywords: system design interview · hld · lld · design interview · design question · scalability · distributed system

System Design interview framework (for SDE-2 and above):

The 5-step approach:

1. Clarify requirements (5 min)
 - Functional: what must the system do?
 - Non-functional: scale, latency, consistency, availability
2. Estimate scale (3 min)
 - DAU, QPS (queries per second), storage needed
 - Example: 10M users × 5 requests/day = ~580 QPS peak
3. High-level design (10 min)
 - Draw: client → API gateway → services → DB → cache → CDN
 - Choose SQL vs NoSQL and justify
4. Deep dive into critical components (15 min)
 - DB schema, indexing strategy, sharding
 - Caching layer (Redis), TTL, eviction policy
 - Async processing (message queue, Kafka)

5. Address bottlenecks & trade-offs (5 min)

- Single points of failure, CAP trade-offs

Must-practice designs:

- URL Shortener (bit.ly)
- Twitter / News Feed
- Rate Limiter
- Notification System (push, email, SMS)
- File Storage (Google Drive / S3)
- Ride Sharing (Uber/Ola)
- Search Autocomplete
- Distributed Cache

■ Resources

- System Design Primer — <https://github.com/donnemartin/system-design-primer>
- Grokking System Design — <https://educative.io/courses/grokking-the-system-design-interview>
- ByteByteGo Blog — <https://blog.bytebytego.com>

INTERVIEW

05. Aptitude & Logical Reasoning

Keywords: *aptitude · logical reasoning · verbal · quantitative · maths · number series · puzzles · campus test*

Aptitude test preparation for campus placements:

Key sections and topics:

Quantitative Aptitude:

- Number system, HCF/LCM
- Percentages, profit & loss, simple/compound interest
- Time, speed & distance; time & work
- Permutations, combinations, probability
- Averages, mixtures, ratios

Logical Reasoning:

- Series completion (number, alphabet, figure)
- Blood relations, seating arrangements
- Coding-decoding, direction sense
- Syllogisms, statement-conclusion, Venn diagrams

Verbal Ability:

- Reading comprehension
- Para jumbles, fill in the blanks
- Error correction, vocabulary

Study plan:

- Practise 30 questions daily for 4 weeks
- Time yourself strictly — 1 min per question target
- Review wrong answers the same day
- Do 3 full mock tests in the last week

■ Resources

- IndiaBix Aptitude — <https://indiabix.com>
- Prepinsta Campus Prep — <https://prepinsta.com>
- Freshersworld Tests — <https://freshersworld.com/aptitude-questions>

Category: Resume

RESUME

06. Resume Building

Keywords: resume · cv · build resume · write resume · resume format · ats · one page

How to build a strong placement resume:

Format rules:

- Single column, clean font (Calibri / Arial / Roboto 10–11pt)
- 1 page for freshers / under 3 years of experience
- No photos, borders, tables, or coloured backgrounds — ATS unfriendly
- Save and submit as PDF

Recommended sections (in order):

1. Name + Contact (email, phone, LinkedIn, GitHub, portfolio)
2. Professional Summary (3 lines: who you are, what you do, what you want)
3. Technical Skills (group by: Languages | Frameworks | Databases | Tools | Cloud)
4. Projects (2–3 projects with Tech stack | Problem | Your contribution | Result)
5. Experience / Internships (reverse chronological)
6. Education (degree, college, CGPA, year)
7. Certifications
8. Achievements (hackathons, ranks, publications)

Writing strong bullet points:

- Formula: [Action verb] + [What you did] + [Result/Impact]
- Bad: 'Worked on a web app'
- Good: 'Built a React + FastAPI web app that reduced manual reporting time by 60%'

ATS optimisation:

- Mirror keywords from the job description
- Use standard section headings (not 'My Achievements' — use 'Achievements')
- Avoid headers/footers — ATS may skip them

■ Resources

- Jobscan — ATS Resume Checker — <https://jobscan.co>
- Resume Worded — <https://resumeworded.com>
- Canva Resume Templates — <https://canva.com/resumes>

RESUME

07. Resume Projects Section

Keywords: *projects · project section · project resume · how to write project · project description*

How to write a strong Projects section on your resume:

Each project entry should have:

- Project Name (bold) | Tech Stack | GitHub link | Live demo (if available)
- 3–4 bullet points describing what you built, how you built it, and the impact

Template:

[Project Name] | React, FastAPI, PostgreSQL, Docker | github.com/you/project

- Built a [what] that [does what] for [who]
- Implemented [specific feature] using [technology] to achieve [result]
- Deployed on [AWS / Heroku / Vercel] with CI/CD pipeline via GitHub Actions
- Achieved [metric] — e.g. 100+ users, 95% uptime, 40% faster load time

Project ideas by role:

SDE: Full-stack CRUD app, URL shortener, real-time chat, REST API service

Data: Data dashboard (Streamlit/Plotly), ML model deployment, ETL pipeline

DevOps: Dockerised app with CI/CD, Kubernetes cluster, infra with Terraform

ML: Image classifier, chatbot, recommendation engine, NLP text classifier

Golden rule: Never put a project you cannot explain in depth. Interviewers will ask you every line of your tech stack.

■ Resources

- GitHub — Open Source Projects — <https://github.com/explore>
- Frontend Mentor — Project Ideas — <https://frontendmentor.io>
- ML Projects — Papers With Code — <https://paperswithcode.com>

RESUME

08. LinkedIn Profile

Keywords: *linkedin · linkedin profile · linkedin tips · linkedin headline · networking linkedin · recruiter*

How to optimise your LinkedIn profile for placement:

Profile photo: Professional headshot, plain background, good lighting

Headline (most important field):

- Do NOT use 'Student at XYZ College'
- Use: 'Final Year CSE Student | Full Stack Developer | React · Python · FastAPI'
- Or: 'Aspiring Software Engineer | DSA | Open to Opportunities 2026'

About section:

- 3–4 lines: who you are, tech stack, what you're building, what you want
- End with: 'Open to full-time SDE roles | Email: you@email.com'

Featured section: Pin your best GitHub project, resume PDF, or portfolio

Experience & Projects: Mirror your resume exactly

Skills: Add 10–20 relevant skills and get endorsements from peers

Activity for visibility:

- Post your project launches, learnings, internship experiences
- Comment thoughtfully on posts by engineers at target companies
- Connect with 2–3 people per day in your target role/company

Open to Work: Enable it (visible to recruiters only) when actively searching

■ Resources

- LinkedIn Profile Tips — <https://linkedin.com/help/linkedin/answer/a549370>
- LinkedIn Learning — Free — <https://linkedin.com/learning>

[RESUME](#)

09. Cover Letter

Keywords: cover letter · covering letter · job application letter · motivation letter

How to write a strong cover letter:**Structure (keep it to 3–4 paragraphs, under 350 words):****Paragraph 1 — Opening:**

State the role you're applying for and where you found it.
Show genuine enthusiasm with 1 specific thing about the company.
Example: 'I'm excited to apply for the SDE role at Razorpay. Your work on building India's payments infrastructure is something I've followed closely.'

Paragraph 2 — Why you:

Pick 2 specific achievements from your resume and connect them to the JD.
Use numbers: 'I built a React app serving 200+ users with a 95% uptime.'

Paragraph 3 — Why them:

Mention 1–2 specific things about the company (product, culture, tech stack).
Show you've done research – don't just say 'reputed company'.

Paragraph 4 — Closing:

Express interest in discussing further and provide contact details.

Mistakes to avoid:

- Don't repeat your entire resume — add colour to 2 points
- Don't use generic openers like 'I am writing to apply for...'
- Don't exaggerate — everything must be verifiable

■ Resources

- Cover Letter Examples — Indeed — <https://indeed.com/career-advice/cover-letter-samples>
- Zety Cover Letter Builder — <https://zety.com/cover-letter-builder>

Category: Backend

BACKEND

10. Python Backend Development

Keywords: *python · python backend · python developer · django · flask · fastapi · python web*

Python backend development roadmap:

Core Python (must know):

- Data types, collections (list, dict, set, tuple)
- OOP — classes, inheritance, dunder methods
- Decorators, context managers, generators
- Error handling (try/except/finally)
- Virtual environments, pip, requirements.txt

Web frameworks comparison:

- FastAPI — modern, async, auto Swagger docs, best for APIs (use this for new projects)
- Django — batteries-included, ORM, admin panel (best for full apps)
- Flask — minimal, flexible (best for small services)

FastAPI essentials:

- Define routes with `@app.get` / `@app.post`
- Use Pydantic models for request/response validation
- Dependency injection with `Depends()`
- Middleware for CORS, auth headers
- SQLAlchemy ORM for database
- JWT auth with `python-jose` + `passlib`

Database skills:

- PostgreSQL — primary relational DB
- SQLAlchemy — ORM with models, sessions, relationships
- Alembic — database migrations

- Redis — caching, rate limiting, sessions

Production checklist:

- Uvicorn / Gunicorn as ASGI server
- Environment variables via .env + python-dotenv
- Docker container for deployment
- Logging + structured error handling

■ Resources

- FastAPI Official Docs — <https://fastapi.tiangolo.com>
- Real Python — <https://realpython.com>
- Django Official Tutorial — <https://docs.djangoproject.com/en/stable/intro/tutorial01>

BACKEND

11. REST APIs

Keywords: *rest api · api design · http · endpoint · request · response · status code · postman*

REST API design essentials:

HTTP methods and when to use them:

- GET — retrieve data (no body)
- POST — create a new resource
- PUT — replace a resource completely
- PATCH — partial update
- DELETE — remove a resource

URL naming conventions:

- Use nouns, not verbs: /users not /getUsers
- Plural for collections: /posts not /post
- Nest for relations: /users/{id}/posts
- Use query params for filtering: /jobs?location=Hyderabad&type;=full-time

HTTP status codes:

- 200 OK, 201 Created, 204 No Content
- 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Validation Error
- 500 Internal Server Error

Request / Response structure:

- Always use JSON
- Consistent error format: { 'detail': 'Error message' }
- Add pagination for list endpoints: { data, total, page, page_size }

Security:

- Bearer token in Authorization header
- Validate all inputs — never trust client data

- Rate limiting to prevent abuse
- HTTPS in production — never HTTP

■ Resources

- RESTful API Design Best Practices — <https://restfulapi.net>
- Postman Learning Center — <https://learning.postman.com>
- HTTP Status Codes — <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

BACKEND

12. Databases & SQL

Keywords: *database · sql · postgresql · mysql · mongodb · nosql · query · join*

Database essentials for backend developers:

SQL fundamentals:

- SELECT, INSERT, UPDATE, DELETE
- JOINS: INNER, LEFT, RIGHT, FULL OUTER
- GROUP BY, HAVING, ORDER BY, LIMIT
- Subqueries and CTEs (WITH clause)
- Window functions: ROW_NUMBER(), RANK(), LAG(), LEAD()

Database design:

- Normalisation: 1NF (atomic values), 2NF (remove partial deps), 3NF (remove transitive deps)
- Primary key, foreign key, unique, not null constraints
- Indexes: B-tree (default), hash, partial — use on frequently queried columns

ACID properties:

- Atomicity: All or nothing
- Consistency: Data stays valid
- Isolation: Concurrent transactions don't interfere
- Durability: Committed data persists

SQL vs NoSQL — when to choose:

- SQL (PostgreSQL, MySQL): structured data, relationships, transactions
- NoSQL (MongoDB): unstructured/flexible schema, horizontal scaling
- Redis: caching, sessions, pub/sub, leaderboards

PostgreSQL-specific:

- JSONB column for semi-structured data
- EXPLAIN ANALYZE to debug slow queries
- psql CLI and pgAdmin for management
- Connection pooling with PgBouncer

■ Resources

- SQLZoo — Practice — <https://sqlzoo.net>

- PostgreSQL Official Docs — <https://postgresql.org/docs>
- Use The Index, Luke — <https://use-the-index-luke.com>

BACKEND

13. Authentication & Authorization

Keywords: *authentication · authorization · jwt · token · session · oauth · auth · login*

Authentication and Authorization guide:

Core concepts:

- Authentication: Who are you? (login with email + password)
- Authorization: What can you do? (role-based access control)

JWT (JSON Web Token):

- Three parts: Header.Payload.Signature (base64 encoded)
- Header: algorithm (HS256), type
- Payload: user ID (sub), expiry (exp), any custom claims
- Signature: HMAC-SHA256(header + payload + secret)
- Store in localStorage (simple) or httpOnly cookie (more secure)
- Short expiry (15 min to 24 hr) + refresh token pattern

Password hashing:

- NEVER store plain text passwords
- Use bcrypt (passlib in Python) — it is slow by design
- `passlib.hash.bcrypt.hash(password)`
- `passlib.hash.bcrypt.verify(plain, hashed)`

OAuth 2.0 (third-party login):

- User clicks 'Login with Google'
- Redirect to Google's auth server → user consents
- Google returns an authorization code
- Your server exchanges code for access token
- Get user profile from Google API

FastAPI auth pattern (this project):

- POST /login → verify password → return JWT
- Axios interceptor attaches Bearer token to every request
- Depends(get_current_user) in FastAPI to protect endpoints

■ Resources

- JWT.io — Debugger — <https://jwt.io>
- OWASP Auth Cheatsheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Passlib Docs — <https://passlib.readthedocs.io>

BACKEND

14. Docker & Containerisation

Keywords: *docker · container · dockerfile · docker-compose · image · containerise · deployment*

Docker essentials for developers:

Core concepts:

- Image: A blueprint (built from a Dockerfile)
- Container: A running instance of an image
- Registry: Where images are stored (Docker Hub, ECR)

Essential Dockerfile for a Python FastAPI app:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Common commands:

```
docker build -t myapp . # Build image
docker run -p 8000:8000 myapp # Run container
docker ps # List running containers
docker exec -it bash # Shell into container
docker logs # View logs
```

Docker Compose (multi-container):

services:

app:

```
build: .
ports: ['8000:8000']
depends_on: [db]
```

db:

```
image: postgres:15
```

environment:

```
POSTGRES_DB: mydb
POSTGRES_PASSWORD: secret
```

Best practices:

- Use `.dockerignore` to exclude `node_modules`, `.env`, `__pycache__`
- Use multi-stage builds to keep images small
- Never copy `.env` into the image — use environment variables

■ Resources

- Docker Official Docs — <https://docs.docker.com/get-started>
- Play with Docker (free) — <https://labs.play-with-docker.com>
- Docker Compose Reference — <https://docs.docker.com/compose/compose-file>

BACKEND

15. Git & Version Control

Keywords: *git · github · version control · branch · merge · rebase · commit · pull request*

Git essentials every developer must know:

Daily workflow commands:

```
git status # See changes
git add # Stage a file
git commit -m 'feat: add login API'
git push origin main
git pull origin main # Sync with remote
```

Branching strategy:

```
git checkout -b feature/login-page # Create + switch
git merge feature/login-page # Merge into main
git branch -d feature/login-page # Delete after merge
```

Undoing mistakes:

```
git restore # Discard unstaged changes
git reset HEAD~1 # Undo last commit (keep changes)
git revert # Safe undo (creates new commit)
git stash # Temporarily save work
git stash pop # Restore stashed work
```

Commit message convention (Conventional Commits):

```
feat: New feature
fix: Bug fix
docs: Documentation
refactor: Code improvement without feature change
test: Add/update tests
chore: Build process, dependencies
```

GitHub best practices:

- Use Pull Requests (PRs) even when working alone — review your own code
- Write meaningful PR descriptions
- Never commit to main directly on shared repos
- Use .gitignore for .env, node_modules, __pycache__, *.pyc

■ Resources

- Pro Git Book (free) — <https://git-scm.com/book/en/v2>
- GitHub Skills — <https://skills.github.com>

- Conventional Commits Spec — <https://conventionalcommits.org>

BACKEND

16. Linux & Command Line

Keywords: [linux](#) · [bash](#) · [terminal](#) · [command line](#) · [shell](#) · [unix](#) · [cli](#) · [server](#)

Linux / Bash essentials for developers:

Navigation:

```
ls -la # List files with permissions
cd /path # Change directory
pwd # Print working directory
find . -name '*.py' # Find files
grep -r 'error' . # Search inside files
```

File operations:

```
cp src dst # Copy
mv src dst # Move / rename
rm -rf dir # Delete recursively (careful!)
chmod +x file # Make executable
chown user file # Change owner
```

Process management:

```
ps aux # List all processes
kill -9 # Force kill process
top / htop # Real-time resource usage
nohup cmd & # Run in background
```

Network:

```
curl -X GET http://localhost:8000
wget # Download file
netstat -tlnp # Ports in use
ssh user@host # Connect to remote server
```

Shell scripting basics:

```
#!/bin/bash
for f in *.py; do python $f; done
if [ -f file ]; then echo 'exists'; fi
```

Useful for DevOps: journalctl (logs), systemctl (services), crontab (schedules)

■ Resources

- Linux Command Cheat Sheet — https://linuxcommand.org/lc3_learning_the_shell.php
- OverTheWire Bandit (practice) — <https://overthewire.org/wargames/bandit>

Category: Frontend

FRONTEND

17. React & Frontend Development

Keywords: *react · jsx · hooks · useState · useEffect · component · vite · frontend*

React essentials for frontend development:

Core concepts:

- Components: function-based, return JSX
- Props: data passed parent → child
- State: local data that triggers re-render (useState)
- useEffect: side effects (API calls, subscriptions, timers)
- useRef: persist values / access DOM without re-render
- useMemo / useCallback: performance optimisation

Project setup (this project uses Vite + React 19):

```
npm create vite@latest my-app -- --template react
cd my-app && npm install && npm run dev
```

React Router (v7):

```
wraps the app
contains all } />
useNavigate() for programmatic navigation
useLocation() to read current route
```

Axios (HTTP client):

```
axios.create({ baseURL }) – create instance
interceptors.request.use – attach auth header automatically
API.get('/profile') – call backend
```

Folder structure (this project):

```
src/pages/ – full page components
src/components/ – shared components (Sidebar, AppLayout)
src/api.js – Axios instance with JWT interceptor
src/App.jsx – routing + auth guards
```

State management pattern used:

- Local useState for page-level state
- localStorage for persistence (prep plan, settings)
- No Redux needed at this scale

■ Resources

- React Official Docs — <https://react.dev>

- Vite Docs — <https://vitejs.dev>
- React Router Docs — <https://reactrouter.com>

Category: Backend

BACKEND

18. FastAPI & SQLAlchemy

Keywords: *fastapi · sqlalchemy · pydantic · orm · model · schema · dependency injection · alembic*

FastAPI + SQLAlchemy pattern used in this project:

Project structure:

```
main.py – FastAPI app, all route handlers
models.py – SQLAlchemy ORM models (DB tables)
schemas.py – Pydantic request/response models
database.py – Engine, SessionLocal, Base, get_db()
auth.py – JWT creation, password hashing, get_current_user()
```

Model → Schema → Route pattern:

```
# 1. SQLAlchemy model (database table)
```

class User(Base):

```
__tablename__ = 'users'
id = Column(Integer, primary_key=True)
email = Column(String, unique=True)

# 2. Pydantic schema (API contract)
```

class UserResponse(BaseModel):

```
id: int
email: str

class Config: from_attributes = True

# 3. Route handler
@app.get('/me', response_model=UserResponse)
```

def get_me(current_user = Depends(get_current_user)):

```
    return current_user
```

Database session dependency:

def get_db():

```
    db = SessionLocal()
    try: yield db
    finally: db.close()
```

Key SQLAlchemy operations:

```
db.query(User).filter(User.email == email).first() # SELECT
```

```
db.add(obj); db.commit(); db.refresh(obj) # INSERT
setattr(obj, 'field', value); db.commit() # UPDATE
db.delete(obj); db.commit() # DELETE
```

■ Resources

- FastAPI Tutorial — <https://fastapi.tiangolo.com/tutorial>
- SQLAlchemy ORM Docs — <https://docs.sqlalchemy.org/en/20/orm>
- Pydantic v2 Docs — <https://docs.pydantic.dev/latest>

BACKEND

19. PostgreSQL & Database Management

Keywords: *postgresql · postgres · psql · pg · database schema · migration · alembic · column*

PostgreSQL setup and management for this project:

Connection (in .env):

```
DATABASE_URL=postgresql://user:password@localhost:5432/dbname
```

psql CLI commands:

```
psql -U postgres # Connect
\l # List databases
\c career_assistant # Switch DB
\dt # List tables
\d student_profiles # Describe table
SELECT * FROM users LIMIT 10; # Query
\q # Quit
```

Adding missing columns (the migration pattern used in this project):

```
ALTER TABLE student_profiles
ADD COLUMN IF NOT EXISTS bio TEXT NULL;
```

- IF NOT EXISTS makes it safe to re-run

Useful queries for this project:

- Check user and profile

```
SELECT u.id, u.email, p.education, p.skills
FROM users u LEFT JOIN student_profiles p ON u.id = p.user_id;
```

- Check resume data

```
SELECT id, filename, status, resume_score FROM resumes;
```

pgAdmin: GUI tool for managing PostgreSQL — recommended for beginners

Backup and restore:

```
pg_dump career_assistant > backup.sql
psql career_assistant < backup.sql
```

■ Resources

- PostgreSQL Docs — <https://postgresql.org/docs/current>

- pgAdmin (GUI) — <https://pgadmin.org>
- SQLAlchemy + Alembic Migrations — <https://alembic.sqlalchemy.org/en/latest/tutorial.html>

Category: CS Fundamentals

CS FUNDAMENTALS

20. DSA — Data Structures

Keywords: *dsa · data structure · array · linked list · tree · graph · heap · hash*

Complete Data Structures reference:

Linear:

- Array: $O(1)$ access, $O(n)$ insert/delete; use for indexed data
- Linked List: $O(1)$ insert/delete at head, $O(n)$ access; use for frequent insert/delete
- Stack: LIFO — push/pop $O(1)$; use for parenthesis, undo, DFS
- Queue: FIFO — enqueue/dequeue $O(1)$; use for BFS, scheduling
- Deque: Double-ended queue; use for sliding window problems

Non-linear:

- Binary Tree: Each node has at most 2 children; use for hierarchy data
- BST: $\text{Left} < \text{root} < \text{right}$; $O(\log n)$ average search
- Heap (Min/Max): Complete binary tree; $O(1)$ min/max, $O(\log n)$ insert
- Hash Map: $O(1)$ average insert/search/delete; use for frequency, caching
- Graph: Nodes + edges; directed/undirected, weighted/unweighted
- Trie: Tree for strings; $O(L)$ search where L = word length; use for autocomplete

Interview pattern → data structure mapping:

- 'Find duplicates' → Hash Set
- 'Find k-th largest' → Min Heap
- 'Shortest path (unweighted)' → BFS (queue)
- 'Balanced parentheses' → Stack
- 'Autocomplete / prefix' → Trie
- 'Range queries on sorted' → Binary Search

■ Resources

- Visualgo — Algorithm Visualiser — <https://visualgo.net>
- NeetCode — Structured Practice — <https://neetcode.io>
- GeeksforGeeks — DS — <https://geeksforgeeks.org/data-structures>

CS FUNDAMENTALS

21. DSA — Algorithms

Keywords: *algorithm · sorting · binary search · two pointer · sliding window · dynamic programming · greedy · dp*

Algorithm patterns every developer must know:

Sorting (know TC for all):

- Merge Sort: $O(n \log n)$ time, $O(n)$ space — stable, divide & conquer
- Quick Sort: $O(n \log n)$ avg, $O(n^2)$ worst — in-place
- Heap Sort: $O(n \log n)$ always — in-place, not stable
- Counting Sort: $O(n+k)$ — use when range is small

Key patterns:

- Two Pointer: Use on sorted arrays — find pair with sum, remove duplicates
 - Sliding Window: Subarray/substring problems — fixed or variable window
 - Binary Search: $O(\log n)$ on sorted arrays; also 'search on answer' pattern
 - BFS: Shortest path in unweighted graphs, level-order tree traversal
 - DFS: Path finding, cycle detection, topological sort
 - Greedy: Make locally optimal choice at each step — activity selection, coin change (specific)
 - Dynamic Programming: Overlapping subproblems + optimal substructure
- Start with memoisation (top-down), convert to tabulation (bottom-up)

Top DP problems to master:

1. Fibonacci (memoised)
2. 0/1 Knapsack
3. Longest Common Subsequence
4. Longest Increasing Subsequence
5. Coin Change
6. Edit Distance
7. Matrix Chain Multiplication

■ Resources

- NeetCode 150 — Algorithm Practice — <https://neetcode.io/practice>
- CP Algorithms — <https://cp-algorithms.com>
- CLRS Algorithms Textbook — <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms>

CS FUNDAMENTALS

22. Object Oriented Programming

Keywords: *oop · object oriented · class · inheritance · polymorphism · encapsulation · abstraction · solid*

OOP concepts for interviews:

The 4 Pillars:

1. Encapsulation: Bundle data + methods in a class; hide internal state
→ Use private attributes + getter/setter methods
2. Abstraction: Expose only what the user needs; hide implementation
→ Abstract classes, interfaces (Java), ABC (Python)
3. Inheritance: Child class inherits parent's properties/methods
→ Promotes code reuse; supports 'is-a' relationships
→ Prefer composition over inheritance for flexibility
4. Polymorphism: Same method name, different behaviour
→ Method overriding (runtime), method overloading (compile-time in Java)

SOLID Principles:

- S — Single Responsibility: One class, one reason to change
- O — Open/Closed: Open for extension, closed for modification
- L — Liskov Substitution: Subclass can replace its parent without breaking code
- I — Interface Segregation: Many specific interfaces > one general interface
- D — Dependency Inversion: Depend on abstractions, not concrete classes

Common Design Patterns:

- Singleton: Only one instance (e.g., DB connection)
- Factory: Create objects without specifying exact class
- Observer: Subscribe to events (e.g., React state, webhooks)
- Strategy: Swap algorithm at runtime (e.g., sort strategy)
- Decorator: Add behaviour without modifying class (e.g., Python @decorator)

■ Resources

- Refactoring Guru — Design Patterns — <https://refactoring.guru/design-patterns>
- SOLID Explained Simply — <https://digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design>

CS FUNDAMENTALS

23. OS & System Concepts

Keywords: operating system · os · process · thread · deadlock · memory · virtual memory · scheduling

Operating Systems concepts for interviews:

Process vs Thread:

- Process: Independent execution unit with its own memory space
- Thread: Lightweight unit within a process; shares memory
- Context switch: OS saves/restores state when switching between processes

Concurrency issues:

- Race condition: Two threads access shared data simultaneously → undefined result

- Deadlock: Two processes each waiting for the other to release a resource
→ Prevent with: lock ordering, timeouts, banker's algorithm
- Mutex: Only one thread at a time (mutual exclusion)
- Semaphore: Counter-based; allows N threads simultaneously

Memory management:

- Stack: Function call frames, local variables (LIFO, auto-managed)
- Heap: Dynamic memory allocation (manual in C, GC in Python/Java)
- Virtual memory: Illusion of large memory using disk paging
- Page fault: Requested page not in RAM → load from disk

CPU scheduling algorithms:

- FCFS, SJF, Round Robin, Priority Scheduling

Common interview questions:

- What happens when you run a program?
- Explain paging and segmentation
- How does Python manage memory? (reference counting + GC)
- What is a context switch overhead?

■ Resources

- OS Concepts — OSTEP (free book) — <https://pages.cs.wisc.edu/~remzi/OSTEP>
- GeeksforGeeks — OS — <https://geeksforgeeks.org/operating-systems>

CS FUNDAMENTALS

24. Computer Networking

Keywords: *networking · tcp · udp · http · https · dns · osi · ip*

Computer Networking essentials for interviews:

OSI Model (7 layers — remember top-down):

7. Application — HTTP, FTP, DNS, SMTP (what users interact with)
6. Presentation — Encryption, compression (SSL/TLS)
5. Session — Session management
4. Transport — TCP (reliable), UDP (fast), ports
3. Network — IP addressing, routing
2. Data Link — MAC addresses, switches
1. Physical — Cables, radio waves

TCP vs UDP:

- TCP: Connection-oriented, reliable, ordered, slower — use for HTTP, email, file transfer
- UDP: Connectionless, unreliable, faster — use for video streaming, gaming, DNS

HTTP/HTTPS:

- HTTP: Stateless application protocol; request/response model

- HTTPS: HTTP + TLS encryption — protects data in transit
- HTTP/2: Multiplexing, header compression, server push
- HTTP/3: Built on QUIC (UDP-based) — lower latency

DNS (Domain Name System):

- Translates domain names → IP addresses
- Resolution order: browser cache → OS cache → recursive resolver → root nameserver

What happens when you type a URL:

1. DNS lookup → IP address
2. TCP handshake (SYN → SYN-ACK → ACK)
3. TLS handshake (for HTTPS)
4. HTTP GET request
5. Server returns HTML
6. Browser parses HTML, fetches CSS/JS, renders page

■ Resources

- Computer Networking — Top-Down (free slides) — https://gaia.cs.umass.edu/kurose_ross/index.php
- How DNS works — Cloudflare — <https://cloudflare.com/en-gb/learning/dns/what-is-dns>

Category: Resume

RESUME

25. Resume

Keywords: resume · cv · ats · format · bullet · action verb · one page · resume tips

Building an ATS-friendly placement resume:

Format rules:

- Single column, no tables or graphics
- Font: Calibri / Arial / Roboto, 10–11pt
- 1 page for freshers
- Save as PDF

Sections (in order):

1. Name + Contact (email, phone, LinkedIn, GitHub)
2. Professional Summary (3 lines)
3. Technical Skills (Languages | Frameworks | Databases | Tools)
4. Projects (2–3 projects with tech stack + impact)
5. Experience / Internships
6. Education
7. Certifications
8. Achievements

Bullet formula:

[Action verb] + [What you built] + [Measurable result]

Example: 'Reduced API response time by 40% through Redis caching'

ATS tips:

- Mirror keywords from job description
- Use standard headings
- No headers/footers — ATS may skip them

Resources

- ATS Resume Checker — Jobscan — <https://jobscan.co>
- Resume Worded — Feedback — <https://resumeworded.com>

Category: CS Fundamentals

CS FUNDAMENTALS

26. DSA

Keywords: *dsa · data structure · algorithm · leetcode · coding · neetcode · competitive*

DSA preparation strategy:

- Start with NeetCode 150 — best curated problem set
- Master first: Arrays, Hash Maps, Two Pointer, Sliding Window, Binary Search
- Then: Trees, Graphs (BFS/DFS), Heap, DP
- Practice timed: 20 min easy / 30 min medium / 45 min hard
- Analyse TC/SC after every problem
- Review wrong answers within 24 hours
- Re-do failed problems after 1 week

Resources

- NeetCode 150 — <https://neetcode.io>
- Striver's DSA Sheet — <https://takeuforward.org/strivers-a2z-dsa-course>
- LeetCode — <https://leetcode.com>

CS FUNDAMENTALS

27. System Design

Keywords: *system design · scalability · distributed · architecture · hld · lld*

System Design interview in 5 steps:

1. Clarify requirements (functional + non-functional)
2. Estimate scale (DAU, QPS, storage)

3. High-level architecture (client → API → DB → cache → CDN)
4. Deep dive into components
5. Address bottlenecks and trade-offs

Must-practice: URL shortener, Twitter feed, rate limiter, notification system, ride-share

Key concepts: horizontal scaling, consistent hashing, SQL vs NoSQL, CAP theorem, message queues

■ Resources

- System Design Primer — <https://github.com/donnemartin/system-design-primer>
- ByteByteGo Blog — <https://blog.bytebytego.com>

Category: Interview

INTERVIEW

28. Interview

Keywords: *interview · mock · hr · behavioural · behavioral · star · round*

Complete interview strategy:

Technical Round:

- Solve 2 LeetCode problems daily for 6 weeks
- Practice coding out loud — interviewers assess communication
- Always state TC/SC for every solution

HR Round (STAR Method):

- S — Situation, T — Task, A — Action, R — Result
- Prepare 8–10 STAR stories: failure, conflict, leadership, tight deadline

Day of interview:

- Clarify constraints before coding
- Start with brute force, then optimise
- Test with examples before submitting

■ Resources

- Pramp — Free Mock Interviews — <https://pramp.com>
- STAR Method — Indeed — <https://indeed.com/career-advice/interviewing/how-to-use-the-star-interview-response-technique>

Category: Career

CAREER

29. Salary

Keywords: salary · negotiate · negotiation · ctc · lpa · offer · package · hike

Salary negotiation guide:

- Research: Glassdoor, Levels.fyi, AmbitionBox, LinkedIn Salary
- Never give the first number
- Counter 10–20% above their offer
- Negotiate the full package: base, bonus, ESOPs, joining bonus, WFH policy
- Get everything in writing before accepting
- Ask for 2–3 days to decide — perfectly normal
- Companies expect negotiation — asking never costs you the offer

■ Resources

- Levels.fyi — <https://levels.fyi>
- AmbitionBox — <https://ambitionbox.com>
- Glassdoor India — <https://glassdoor.co.in/Salaries>

CAREER

30. Skills

Keywords: skill · learn · upskill · roadmap · python · react · cloud · devops

High-demand skills for software roles in 2026:

Core (all developers):

- DSA + System Design
- SQL + one NoSQL DB
- Git + REST API design

Frontend: React, TypeScript, CSS, Web Performance

Backend: Python/Node.js, FastAPI/Express, PostgreSQL, Redis, Docker

Cloud: AWS fundamentals (EC2, S3, RDS, Lambda)

AI/ML: Python, Pandas, Scikit-learn, PyTorch, LLM APIs

Fastest growing in 2026: TypeScript, Rust, LLM prompt engineering, Kubernetes

■ Resources

- roadmap.sh — <https://roadmap.sh>
- The Odin Project (free) — <https://theodinproject.com>

CAREER

31. Placement

Keywords: placement · campus · off-campus · job search · fresher · 2025 · 2026

Placement strategy for freshers (2025–2026):

6 months before:

1. Finalise target role (SDE / DA / DevOps)
2. Resume: 1 page, ATS-friendly
3. DSA: 100+ problems minimum
4. Build 2–3 projects with GitHub + live demo

3 months before:

5. Apply: LinkedIn, Naukri, Instahyre, company portals
6. Mock interviews: 2/week minimum
7. Research each company before every round

Day before interview:

8. Revise resume line by line
9. Prepare STAR stories
10. Sleep early

■ Resources

- LinkedIn Jobs — <https://linkedin.com/jobs>
- Naukri.com — <https://naukri.com>
- Instahyre — <https://instahyre.com>

CAREER

32. Study Plan

Keywords: study plan · preparation plan · schedule · week · 4 week · plan · timetable

12-week placement preparation plan:

Week 1–3: DSA Foundations

Arrays, Strings, Linked Lists, Stacks, Queues, Hash Maps

Week 4–6: Algorithms

Sorting, Binary Search, Two Pointer, Sliding Window, Recursion

Week 7–8: Advanced DSA

Trees, Graphs, Heaps, Dynamic Programming

Week 9–10: System Design

HLD, Scalability, Caching, Databases, 5 core designs

Week 11–12: Interview Readiness

STAR stories, mock interviews, company research, resume polish

Daily: 2–3 hr DSA + 30 min review + 30 min theory

■ Resources

- NeetCode Roadmap — <https://neetcode.io/roadmap>
- Striver's SDE Sheet — <https://takeuforward.org>

CAREER

33. Career Switch

Keywords: career switch · career change · transition · change role · new field

Switching careers or roles:

Step 1: Define your target role precisely

Step 2: Identify skill gaps (use the Skill Gap page)

Step 3: Build a transition portfolio

- 1–2 projects in the new domain
- Relevant certifications (AWS, Google, Meta)

Step 4: Update LinkedIn headline to target role

Step 5: Network — 70% of roles are filled via referrals

Step 6: Prepare a positive 'why switch' narrative

Step 7: Target companies that value cross-functional backgrounds

■ Resources

- LinkedIn Learning — <https://learning.linkedin.com>
- Coursera Careers — <https://coursera.org>

CAREER

34. Open Source Contribution

Keywords: open source · contribute · github contribution · pull request · open source project · hacktoberfest · gsoc

How to start contributing to open source:

Why it matters:

- Real-world codebase experience beyond college projects
- Signals initiative and coding ability to recruiters
- Builds your GitHub profile (green squares count)
- Networking with professionals globally

Where to start (beginner-friendly):

1. GitHub's good-first-issue label: github.com/explore/topics/good-first-issue
2. Up For Grabs: up-for-grabs.net

3. CodeTriage: codeltrriage.com
4. First Contributions: firstcontributions.github.io

Contribution workflow:

1. Fork the repo
2. Clone your fork locally
3. Create a branch: `git checkout -b fix/typo-in-readme`
4. Make your change + commit with meaningful message
5. Push and open a Pull Request
6. Respond to reviewer comments

Programs to participate in:

- Hacktoberfest (October — free T-shirt for 4 PRs)
- Google Summer of Code (GSoC) — paid internship
- GirlScript Summer of Code (GSSoC)
- MLH Fellowship

■ Resources

- First Contributions (tutorial) — <https://firstcontributions.github.io>
- GitHub Explore — Good First Issues — <https://github.com/explore/topics/good-first-issue>
- GSoC Official — <https://summerofcode.withgoogle.com>

CAREER

35. Certifications

Keywords: *certification · certify · aws certification · google cert · cloud cert · meta cert · certificate course*

High-value certifications for software roles:

Cloud (most in demand):

- AWS Cloud Practitioner (entry level, 2–4 weeks prep)
- AWS Solutions Architect – Associate (3–6 months prep)
- Google Associate Cloud Engineer
- Microsoft AZ-900 (Azure Fundamentals)

Development:

- Meta Front-End Developer Certificate (Coursera)
- Meta Back-End Developer Certificate (Coursera)
- Google Data Analytics Certificate
- IBM Data Science Professional Certificate

DevOps:

- Docker Certified Associate
- Certified Kubernetes Administrator (CKA)
- HashiCorp Terraform Associate

AI / ML:

- TensorFlow Developer Certificate
- DeepLearning.AI Specialisation (Coursera)
- Hugging Face NLP Course (free)

Free options:

- AWS Skill Builder (free tier)
- Google Cloud Skills Boost (free quests)
- freeCodeCamp certifications
- NPTEL certifications (IIT-backed, widely recognised in India)

■ Resources

- AWS Skill Builder — <https://skillbuilder.aws>
- Coursera Professional Certs — <https://coursera.org/professional-certificates>
- NPTEL Online Courses — <https://nptel.ac.in>
- freeCodeCamp — <https://freecodecamp.org/learn>

CAREER

36. Freelancing & Remote Work

Keywords: *freelance · freelancing · remote work · upwork · fiverr · contract · gig · work from home*

Getting started with freelancing as a developer:

Platforms to find work:

- Upwork: Best for long-term contracts (web, backend, AI projects)
- Fiverr: Good for one-time gigs (small websites, APIs, automation)
- Toptal: High-paying but very selective (top 3% of applicants)
- Freelancer.in: India-focused, lower competition
- LinkedIn: Direct outreach to startups needing part-time devs

Building your freelancer profile:

1. Pick a niche: 'Full Stack (React + FastAPI)' or 'Python Data Automation'
2. Build 2–3 portfolio projects and deploy them live
3. Write a clear profile with your stack and what problems you solve
4. Start with lower rates to get initial reviews, then increase

High-demand freelance skills in 2026:

- React + Next.js web apps
- Python automation and scraping
- FastAPI / Django REST backends
- LLM integrations (ChatGPT API, Ollama)
- Mobile apps (React Native, Flutter)

Tax and legal in India:

- Income above ₹2.5L per year is taxable
- File ITR-4 (presumptive taxation for freelancers)
- Consider registering as a sole proprietor for larger contracts

■ Resources

- Upwork — <https://upwork.com>
 - Toptal — <https://toptal.com>
 - Fiverr — <https://fiverr.com>
-

AI Career Assistant

Your AI-powered placement companion for students and early-career professionals.

© 2026 AI Career Assistant · All career guidance is for informational purposes.